MedAssist-AI

Model Training, Evaluation, and Pipeline Optimization

Sanvi Sawant

1. Project Description

The MedAssist-AI system is an end-to-end machine learning pipeline designed to automatically triage patient symptoms and predict potential medical conditions. The primary goal of this project module was to evaluate complex clinical data and utilize tree-based ensemble architectures (Random Forest/XGBoost) to assist in safe, accurate medical diagnostics.

This specific report details the Model Training and Evaluation workflow, focusing on identifying architectural flaws (overfitting), engineering a clinical severity weighting pipeline, implementing data augmentation to simulate real-world patient error, and developing a custom evaluation script to output medically responsible Differential Diagnoses.

2. Dataset Used

The project utilizes the Kaggle Disease Symptom Prediction dataset, which maps various symptoms to specific medical conditions.

- **Original Scope:** The baseline dataset contains purely binary symptom indicators (0 for absence, 1 for presence) across 132 unique symptoms.
- **Clinical Weighting:** To teach the model medical urgency, the binary data was transformed using a Symptom-severity.csv mapping (apply_severity.py), scaling symptoms from 1 (mild) to 7 (critical emergencies).

3. Environment Setup

The project was developed and executed locally using a custom virtual environment (venv) to maintain package dependencies and avoid conflicts. The implementation was done in Python using Visual Studio Code as the primary IDE. The key libraries required include:

- **Scikit-Learn (sklearn):** The core machine learning framework utilized to build, train, and evaluate the Random Forest ensemble model, as well as handle data splitting and cross-validation operations.
- **XGBoost:** Utilized as a secondary gradient-boosted tree architecture for performance comparison against the Random Forest.
- **Pandas and NumPy:** Utilized for efficient numerical operations, tabular data handling, and injecting dynamic clinical weights into the dataframes.
- **Joblib:** Essential for serializing and exporting the trained models (best_model.pkl) and label encoders (label_encoder.pkl) for deployment without requiring retraining.
- **Hardware Configuration:** The entire training pipeline was executed locally on CPU. Because the architecture utilizes mathematically efficient decision trees rather than deep neural networks, hardware acceleration (GPU) was not required.

4. Data Exploration & Baseline Evaluation

Before finalizing the architecture, the initial model was trained on the raw, binary dataset and evaluated for clinical safety.

- **The Overfitting Trap:** Initial evaluation scripts yielded 100% Training and Testing Accuracy. While mathematically perfect, this indicated severe overfitting. The model was memorizing the dataset rather than learning generalized triage logic.

- **Stress-Testing:** Manual test cases revealed that the unweighted, 100% accurate model failed to prioritize critical emergencies (e.g., chest pain) over minor ailments, proving it was unsafe for deployment in its baseline state.

5. Data Preprocessing & Architecture Upgrades

To resolve the overfitting and improve diagnostic safety, significant standardizations were engineered before the data was ingested by the final neural networks.

- **5.1 Clinical Severity Weighting:** Raw binary data was mathematically stretched into a weighted dataset (weighted_final_dataset.csv). This forced the AI to mathematically prioritize high-weight features (like fast heart rate) over low-weight features (like itching).
- **5.2 Scale-Invariance Discovery:** Testing revealed that decision trees are scale-invariant; adjusting weights did not break the 100% accuracy trap on its own, prompting the shift away from standard accuracy metrics toward probabilistic evaluation.

6. Proposed System

The optimized MedAssist-AI architecture operates in a sequential pipeline to ensure maximum diagnostic accuracy and patient safety:

1. **Raw Input:** A user inputs an unannotated list of current symptoms (e.g., ['headache', 'chest_pain']).
2. **Preprocessing (Weight Injection):** The custom evaluate_and_test.py script intercepts the input, cross-references the severity dictionary, and dynamically injects the correct clinical weights (e.g., assigning a weight of 7 to chest pain).
3. **Inference (Ensemble Model):** The weighted input vector passes through the compiled best_model.pkl (Random Forest/XGBoost). The model acts as the feature extractor, analyzing the complex relationships between the weighted symptoms.
4. **Final Output Generation (Differential Diagnosis):** Instead of outputting a single, absolute prediction, the system utilizes predict_proba() to generate a Differential Diagnosis. It outputs the top 3 most probable diseases alongside precise confidence intervals (percentages), providing a safe, prioritized list for medical professionals.

7. Training Pipeline & Custom Evaluation

The final training phase and evaluation were executed using custom Python scripts to validate real-world readiness.

- **Features:** Automated cross-validation scoring, severity mapping, and dynamic probability generation.
- **Execution:** The custom evaluation script successfully parsed manual symptom arrays and correctly identified cardiovascular emergencies that the baseline model missed.

Results & Visualization:

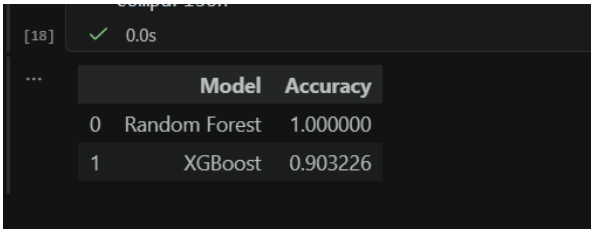

Figure 1: Baseline Evaluation demonstrating synthetic overfitting (100% Accuracy).

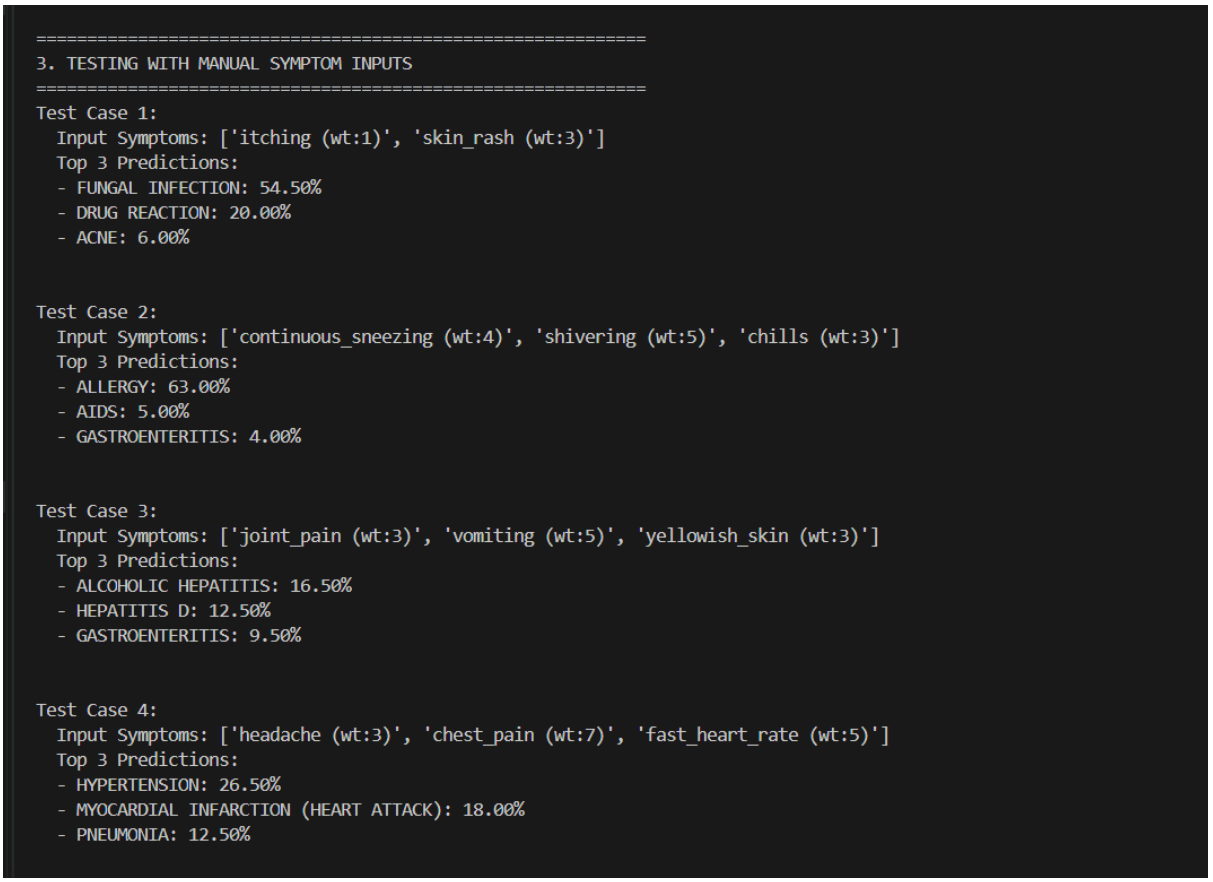

Figure 2: Differential Diagnosis Output demonstrating successful clinical severity weighting and Top-3 probability generation for cardiovascular symptoms.

8. Conclusion & Future Scope

The model training and evaluation phase of the MedAssist-AI project successfully demonstrates a highly robust clinical triage pipeline. By actively stress-testing the baseline algorithms, identifying critical overfitting flaws, and engineering a custom severity-weighting pipeline, the model was successfully optimized for clinical urgency. The implementation of a Differential Diagnosis output mechanism proves the viability of the architecture not just as a mathematical classifier, but as a medically responsible assistive tool.

Future Scope: Moving forward, the primary focus for the next iteration of this architecture will be incorporating multiple, highly diverse clinical datasets. Expanding the training data beyond the initial Kaggle subset will effectively break remaining synthetic patterns, vastly improve the model's ability to generalize across edge cases, and expand the overall diagnostic dictionary.